# Control de velocidad de un motor DC mediante PID

Informe del desarrollo experimental con ESP32 S3

Rafael Daguer Puello | Control Inteligente | Septiembre de 2026

**Resumen**

Se desarrolló un prototipo para regular la velocidad de un motorreductor DC mediante un controlador PID ejecutado en una ESP32 S3. El montaje integra un encoder óptico, un driver L298N y una interfaz HTML que permite cambiar la referencia y las ganancias, así como observar la velocidad y la acción de control. El proyecto evolucionó desde una simulación visual hasta la supervisión del motor físico.

El trabajo permitió implementar la adquisición por interrupciones, el mando PWM y la comunicación con la interfaz. La principal dificultad fue la variabilidad de la señal del encoder, asociada a problemas de alineación y posibles perturbaciones eléctricas. Tras ensayar el funcionamiento sin filtros digitales, se restauró la versión con rechazo de flancos próximos, mediana y suavizado. El prototipo constituye una plataforma de demostración del PID; la precisión absoluta y su desempeño dinámico aún requieren validación experimental independiente.

## Objetivo general

Implementar y visualizar un control de velocidad en lazo cerrado para un motor DC, relacionando el error de velocidad con las acciones proporcional, integral y derivativa, y evaluando las dificultades reales de medición y accionamiento.

## Alcance del desarrollo

Se abordaron la selección del sensor, la adaptación de niveles eléctricos, las pruebas de conteo y PWM, la implementación del PID y la transformación de la interfaz de simulación en una herramienta para el montaje físico. El control se realiza en un sentido de giro. La consigna disponible en la interfaz va de 0 a 200 RPM y la señal PWM se presenta de 0 a 100 %.

## Organización del sistema

La interfaz envía la referencia y las ganancias por comunicación serial USB. La ESP32 compara la consigna con las RPM calculadas a partir del encoder, determina la salida del PID y modifica el PWM del L298N. El motor mueve la rueda ranurada, cerrando el lazo de realimentación. La telemetría regresa a la interfaz para su representación gráfica.

# Montaje y acondicionamiento de señal

| Elemento | Función en el prototipo |
|---|---|
| Motorreductor DC amarillo | Planta cuya velocidad se desea regular. |
| ESP32 S3 N16R8 | Adquisición, cálculo del PID, PWM y comunicación USB. |
| Encoder óptico y rueda de 20 ranuras | Detección de las transiciones de paso y bloqueo de luz. |
| Driver L298N | Etapa de potencia entre el controlador y el motor. |
| Fuente de 12 V y convertidor reductor | Alimentación con reducción para los circuitos de baja tensión. |
| Potenciómetro de 10 kΩ | Entrada para introducir una perturbación en el mando. |
| Resistencias y capacitores cerámicos | Adaptación del nivel lógico y reducción de ruido. |

### Conexiones de control

Se asignó GPIO4 a ENA del L298N, retirando el puente de habilitación para permitir el PWM. IN1 permanece a 5 V e IN2 a GND, fijando el sentido de giro. El encoder se conecta a GPIO7 a través del divisor resistivo. El cursor del potenciómetro se conecta a GPIO8 y sus extremos a 3,3 V y GND. Las referencias de tierra de los circuitos deben estar unidas.

### Alimentación y adaptación del encoder

Se descartó la fuente de 5 V y 1 A y se conservó la de 12 V y 2 A con un convertidor reductor. Los 12 V de la fuente no representan la tensión aplicada directamente al motor amarillo. La caída de tensión del L298N también influye en el margen de accionamiento disponible.

La salida del encoder presentó aproximadamente 0,14 V sin barrera y 4,8 V con barrera. Se incorporó un divisor para llevar el nivel alto al dominio de entrada de la ESP32. Se evaluaron resistencias de 10 kΩ en la rama superior y 33 kΩ o 20 kΩ hacia tierra. Para una salida ideal de 4,8 V, se obtienen 3,68 V y 3,20 V, respectivamente; el usuario reportó finalmente entre 2,9 y 3,0 V. La carga del módulo hace necesario comprobar el voltaje real, no únicamente el cálculo ideal.

### Filtros físicos instalados

Se emplearon capacitores cerámicos: 100 nF, código 104, directamente entre los terminales del motor; otro de 100 nF entre alimentación y tierra del encoder, próximo al módulo; y 10 nF, código 103, entre la señal adaptada y tierra. Su propósito es reducir perturbaciones eléctricas. Su instalación no elimina por sí misma las irregularidades mecánicas de la rueda.

# Medición de velocidad y controlador PID

## Conteo por interrupciones

La rueda tiene 20 ranuras. Inicialmente se contaba un tipo de flanco, equivalente a 20 eventos por vuelta. Por recomendación del profesor se cambió la interrupción a CHANGE, contando tanto la entrada como la salida de cada ranura: 40 flancos por revolución. Este cambio aumenta la resolución del conteo; requiere actualizar el factor de conversión para evitar duplicar las RPM indicadas.

El programa atiende los flancos mediante una interrupción y acumula el contador. Cada 250 ms aproximadamente toma una copia protegida del conteo y calcula la velocidad usando el tiempo realmente transcurrido. No espera a ese instante para leer cada ranura.

$RPM = 60 \times N / (40 \times \Delta t)$
N es el número de flancos aceptados y Δt es el intervalo de medición en segundos. Para $\Delta t = 0{,}25$ s, cada flanco representa 6 RPM; por ejemplo, 20 flancos corresponden a 120 RPM.

## Filtrado de la medición

La versión restaurada v3.0 rechaza flancos separados por menos de 100 µs del último aceptado. Después aplica una mediana de tres muestras y un suavizado exponencial con $\alpha = \Delta t / (0{,}15 + \Delta t)$. La derivada de la velocidad también se suaviza. El filtrado reduce variaciones rápidas, pero añade retardo y puede ocultar información si se ajusta de forma excesiva. La salida bruta y el número de descartes permiten observar su efecto.

## Ley de control implementada

El error se define como e = referencia − velocidad medida. La acción proporcional es $P = Kp \times e$. La integral acumula el error en el tiempo y produce $I = Ki \times$ acumulación. La derivativa se calcula sobre la medición, $D = -Kd \times$ derivada de la velocidad, para reducir el salto derivativo cuando cambia la referencia. La suma P + I + D determina el mando antes de aplicar la perturbación y la saturación.

La salida interna se limita al intervalo 0–255, equivalente a 0–100 % en la interfaz. La integral utiliza integración condicional: evita acumular error cuando este incrementaría una saturación. Los valores iniciales son $Kp = 0{,}15$, $Ki = 0{,}20$ y $Kd = 0{,}005$; son parámetros de partida, no una sintonización óptima demostrada.

Se eliminaron el impulso automático de arranque y el PWM mínimo obligatorio para que las acciones del PID fueran observables. Cada ganancia puede anularse de manera independiente; con las tres en cero, la salida de control queda en cero. La referencia cero ordena la parada y reinicia el controlador. Se conserva un diagnóstico de conteo superior a 1000 RPM para señalar una medición anómala.

# Interfaz y pruebas experimentales

## Supervisión del motor físico

La interfaz HTML inicial se adaptó para priorizar el montaje real y mantener la simulación en una página separada. Se conservaron la representación del motor y los controles, dando mayor presencia a las gráficas de velocidad, PWM y error. La consigna y las ganancias se aplican al mover los deslizadores, sin un botón adicional de aplicar. La referencia inicial es cero y el botón de restablecimiento recupera los parámetros predeterminados.

La conexión serial permite recibir velocidad, conteos, términos del PID y estado del sistema. La interfaz comprueba que existan datos recientes antes de habilitar el arranque. Durante la puesta en marcha se corrigieron problemas de cableado en las salidas del motor y de configuración de la comunicación USB en Arduino IDE.

## Respuesta observada en las pruebas de PWM

Antes del PID se variaron valores de PWM para verificar el accionamiento. La siguiente tabla resume una secuencia registrada con la conversión original de 20 pulsos por vuelta. Los rangos describen las lecturas del programa; no constituyen una calibración de la velocidad real.

| PWM interno | Ciclo útil | RPM indicadas |
|---|---|---|
| 255 | 100,0 % | 363–399 |
| 200 | 78,4 % | 333–399 |
| 150 | 58,8 % | 318–360 |
| 100 | 39,2 % | 270–369 |
| 75 | 29,4 % | Valores variables y parada |

En general, la velocidad indicada disminuyó al reducir el PWM, aunque las lecturas se solaparon. Con PWM 75 apareció un pico de 642 RPM y posteriormente se registraron ceros, coincidiendo con una parada espontánea. También se observó que PWM 100 no permitía arrancar en una prueba. Esto muestra que el arranque y el mantenimiento del giro no tienen necesariamente el mismo umbral de mando.

## Influencia del encoder

Se reportó desalineación de la rueda y oscilaciones importantes en la lectura. En una configuración anterior de filtrado, valores brutos próximos a 400–500 RPM se redujeron a aproximadamente 150–156 RPM, con numerosos eventos descartados. La diferencia mostró la necesidad de revisar el filtro antes de interpretar la velocidad como un dato confiable.

Después de instalar filtros físicos se retiró temporalmente el filtrado de software. Al reportarse problemas, se restauró la versión anterior. Esta decisión recupera la configuración conocida, pero no demuestra por sí sola que la medida corresponda exactamente a la velocidad mecánica.

# Discusión y conclusiones

## Interpretación de los resultados

El desarrollo mostró que la calidad de la realimentación condiciona la respuesta del PID. Un conteo falso puede producir una aparente sobrevelocidad y reducir bruscamente el PWM; la pérdida o el rechazo excesivo de flancos puede provocar el efecto contrario. Por ello, una curva visualmente estable después del filtrado no basta para demostrar precisión.

El motorreductor y la etapa de potencia también introducen restricciones. La fricción y el umbral de arranque explican que un mando suficiente para mantener el giro pueda no iniciar el movimiento desde reposo. Si la consigna supera la velocidad alcanzable, el controlador satura. La saturación debe distinguirse de un error de programación o de sintonización.

## Perturbación mediante potenciómetro

El potenciómetro introduce una reducción programada de hasta 60 unidades del mando PWM antes de la saturación. Esto permite observar si el controlador compensa una pérdida de acción del actuador. No aplica un torque resistente al eje: una prueba de carga mecánica requeriría un freno, una carga acoplada u otro mecanismo medible. Ambas experiencias son útiles, pero representan perturbaciones diferentes.

## Limitaciones de la evaluación

No se dispone de una medición independiente de RPM ni de una campaña final de escalones con condiciones repetibles. Por ello, no se atribuyen al prototipo valores demostrados de error estacionario, sobreimpulso, tiempo de establecimiento o precisión absoluta. La desalineación, la resolución de 6 RPM por flanco en ventanas de 250 ms y el retardo de los filtros deben considerarse al interpretar las gráficas.

## Conclusiones

Se implementó una plataforma que integra lectura óptica, PWM, PID y supervisión gráfica del motor físico. El cambio a 40 flancos por vuelta aumentó la resolución de adquisición y mantuvo el conteo mediante interrupciones. La interfaz facilita la experimentación al aplicar inmediatamente la consigna y las ganancias y mostrar el mando en porcentaje.

La revisión del controlador permitió separar las acciones P, I y D de ayudas de arranque y mínimos de PWM que dificultaban su interpretación. Los ensayos confirmaron la importancia de estudiar en conjunto el sensor, el acondicionamiento de señal y el actuador. El estado final del código corresponde a la versión v3.0 con filtrado de software restaurado.

Para cuantificar el desempeño, el siguiente paso es alinear y fijar la rueda, contrastar las RPM con un instrumento independiente y registrar respuestas a escalones dentro del rango alcanzable. Con esa base podrá ajustarse el PID y comparar de forma objetiva la respuesta con y sin perturbaciones.

## Material del proyecto

Firmware LecturaEncoder.ino y PidCore.h; interfaz simulador_motor_dc.html y montaje.js; registros seriales y observaciones de las pruebas de montaje. Las cifras experimentales de este informe corresponden a las lecturas registradas durante el desarrollo.

Repositorio en GitHub: https://github.com/rafadaguer14-ctrl/Control-Inteligente.git